LegalSpan coarse spans and legal roles

CS781 Phase 2 legal named entity recognition | source-audited report

Task and evidence

Predict exact original-character entity spans in 12 legal categories. The source archive holds the actual code, configuration, run record, selected weights, and prediction ZIP. This report was produced after training from those records; it is not an executed historical notebook.

Architecture

This warm-started LegalSpan variant first proposes generic spans, then classifies each span into one of six legal roles from boundary, pooled, header, and length features. It also contains a repeated-name consistency mechanism.

Path	Legal text + header -> Warm-start LegalSpan -> Coarse BIO tags -> CRF span proposal
Output	Span feature pooling -> Six legal roles -> Consistency + export

Training and validation

Method: Coarse span + six-role classifier. This export's archived held-out strict macro-F1 is 0.408721 on the 12-document local validation split. Its selected checkpoint and prediction ZIP hashes are in evidence/run_summary.json. The official test set has 35 document IDs; no hidden Codabench score is established by this archive.

Implementation map

Key original files: role_model.py, run_role_span.py, boost_model.py. The companion notebook explains the tensor route and reads archived evidence without presenting invented execution outputs.

Reproducibility boundary

The package includes exact source snapshots and small run records, plus the archived prediction ZIP. Large model weights, tokenizers, and course data stay in the separate complete archive. Original machine paths in config files need remapping for a new environment. The consensus rule was selected on validation; 100M/200M D222 exports were completed after the original deadline where marked.

Interpretation

Model variants change either pretrained backbone, parameter-update method, NER head, or pretrained checkpoint. A score difference does not isolate one factor unless all other choices match. The local held-out set is small, so its ranking should not be described as a hidden-test result.

Recorded model and adaptation

The source and run records together identify the following settings. Counts include the Phase 2 NER head. The method changes which backbone parameters are updated; it does not change the original Phase 1 tokenizer during this run.

Field	Archived value
Base family	LegalSpan
Adaptation	Coarse span + six-role classifier
Total parameters	113001636
Trainable parameters	112995211
Backbone LR	5e-06
Head LR	0.0003
Pretrained SHA-256	2eb64c8489aa4ead38ec…
Selected NER SHA-256	5123b67cb16a531d0b04…

Training signal and exact-span evaluation

For the common NER runner, the character tagger minimizes weighted cross-entropy over 25 BIO categories; A0 also matches its verified B1 teacher with temperature-2 KL. D222 adds CRF negative log-likelihood and a weighted boundary objective. HeaderCRF combines cross-entropy, CRF, and boundary losses. The validation scorer counts a prediction as correct only when the start, end, and label all match the gold span; macro-F1 averages the 12 per-label F1 values equally.

Observed optimization trace

These five entries are sampled from the archived training log; loss is a per-batch training value, not held-out macro-F1. Its decline alone cannot establish improved entity extraction.

Optimizer step	Epoch	Logged batch loss
1	1	0.5851
210	2	0.3162
420	3	0.3245
630	4	0.3138
830	5	0.2974

Label-level held-out scores

The following F1 values come from the archived selected validation prediction. Low-frequency labels are sensitive to a few documents, so read them beside their support and error counts in evidence/selection.json.

Entity label	F1	TP	FP	FN
APP	0.105	11	89	98
RESP	0.037	3	59	98
A.COUNSEL	0.655	36	29	9

Entity label	F1	TP	FP	FN
R.COUNSEL	0.393	12	18	19
JUDGE	0.480	18	22	17
WIT	0.000	0	61	21
AUTH	0.215	47	185	158
COURT	0.680	141	64	69
STAT	0.553	167	93	177
PREC	0.266	71	201	191
DATE	0.937	313	23	19
CASENO	0.584	80	56	58

Verification and reproducibility

The prediction ZIP has a single root prediction.json, passed CRC, and contains exactly 35 official test IDs. Its SHA-256 was compared with the completed run record. Each copied code file has a SHA-256 listed in evidence/SOURCE_HASHES.json. Exact re-execution requires the original selected weights, tokenizer and course data preserved in the complete archive; this compact package does not embed them or assert that historical notebook output cells existed.